Sumário

- Estruturas de Repetição
- Listas
- Strings

Estruturas de Repetição

É comum que em um algoritmo, uma mesma instrução precise ser executada várias vezes seguidas. Para facilitar a programação, normalmente utilizamos um loop (ou estrutura de repetição) que permite executar o mesmo bloco de instruções repetidas vezes. Em Python, temos algumas estruturas de repetição:

- while
- for

Estruturas de Repetição

WHILE

A estrutura While tem como objetivo repetir um conjunto de comandos enquanto uma condição for verdadeira. A condição imposta sempre é verificada no início da estrutura e, enquanto for satisfeita, o While executa todos os comandos que estão dentro do loop.

```
num = int(input("Informe o ultimo numero a ser impresso: "))
cont = 2

while cont <= num:
        print(cont)
        cont = cont +2
```

Estruturas de Repetição

FOR

O laço for é usado quando se tem um bloco de código que será repetido um número fixo de vezes.

```
for i in range(10): #i recebe de 0 a 9, incrementando 1 a cada vez
        print(i) #imprime i
```

Estruturas de Repetição

FOR

A função *range()* nos permite também definir o intervalo do loop, ou seja, o número inicial e final da repetição, além do step, que é o intervalo entre cada elemento.

```
for i in range(1,10,2):
        print(i)
```

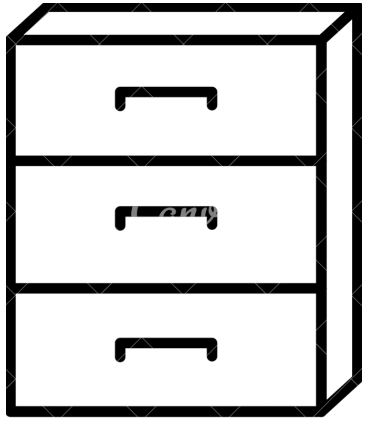
Estruturas de Repetição

FOR

A função “break”, que tem como objetivo terminar de forma abrupta uma repetição. Por exemplo, se estivermos dentro de uma repetição e um determinado resultado ocorrer, o programa deverá sair do loop e continuar na primeira linha seguinte à estrutura.

```
soma = 0

for i in range(5):
        n = int(input("Informe um numero inteiro para soma ou 0 para
sair: "))
        if n == 0:
                break
        soma = soma + n

print("A soma dos numeros digitados é: %d" %soma)
```

Listas



O QUE SÃO?

Listas são um tipo de variável que armazena várias informações ao mesmo tempo.

As informações armazenadas nas listas podem ser de vários tipos e cada informação é separada por uma vírgula. Todas as informações são armazenadas dentro de um colchete.

```
v1 = []
v2 = [1,2,3]
v3 = ["oi", 2.3, 3]
```

Listas

MODIFICANDO UM ITEM DA LISTA

Podemos imaginar um programa onde são armazenadas as notas de todos os alunos. Para modificar algum valor dessa lista, basta informar o nome da lista, junto com o índice da nota que deseja acessar, e igualar a nova nota que deseja armazenar.

```
notas = [6, 5, 7.5, 8, 5.5]

print("Antes de modificar: %d" %notas[1])

notas[1] = 9

print("Depois de modificar: %d" %notas[1])
```

```
File  Edit  View  Search  Terminal  Help
amanda@unito:~$ python3 ./Desktop/exemplo2Listas.py
Antes de modificar: 5
Depois de modificar: 9
amanda@unito:~$
```

Listas

COPIA DE UMA LISTA

Para copiar os dados de uma lista devemos ter bastante atenção. Isso porque se somente igualarmos duas listas, não estaremos copiando os dados da lista, e sim seu endereço de memória. Dessa forma, teremos uma mesma lista com dois nomes diferentes.

Ao usarmos o comando L[:], é criada uma cópia da lista para um endereço de memória diferente, permitindo que os dados dessas listas sejam modificados de forma independente.

```
L = [1, 2, 3, 4, 5]

V = L            # Forma errada
V = L[:]         # Forma certa
```

Listas

FATIAMENTO DE UMA LISTA

Para copiar só uma parte de uma lista usamos o mesmo comando, informando o início e o fim dos índices que se deseja copiar.

```
L = [1, 2, 3, 4, 5]

V = L[:]
V2 = L[2:]      # Criando uma cópia do índice 2 até o final da lista
V3 = L[:3]      # Criando uma cópia até o índice 3

print("Lista V: ")
print(V)

print("\nLista V2:")
print(V2)

print("\nLista V3:")
print(V3)
```

```
Lista V:
[1, 2, 3, 4, 5]

Lista V2:
[3, 4, 5]

Lista V3:
[1, 2, 3]
```

Listas

PERCORRENDO UMA LISTA

Para percorrermos uma lista devemos usar as estruturas de repetição *for* ou *while*.

Exemplo: podemos calcular a nota média da turma:

```
notas = [6, 5, 7.5, 8, 5.5]
soma = 0

for i in range(5):
        soma = notas[i] + soma

media = soma/5

print("Media das notas: %d" %media)
```

Listas

PERCORRENDO UMA LISTA

Quando não soubermos a quantidade de elementos de uma lista, podemos usar o comando *len()*. Ele informa o tamanho da nossa lista.

```
L = [1,2,3,4,5]
t = 0

while t < len(L):
        print(L[t])
        t = t+1
```

Listas

ADICIONANDO ELEMENTOS EM UMA LISTA

Uma das principais vantagens de utilizar-se uma lista é poder adicionar quantos elementos quiser a qualquer hora. Para adicionar um elemento ao final da lista basta utilizar o comando append(). Podemos também adicionar uma lista em outra através do comando extend().

```
L = [1,2,3,4,5]
V = [7,8,9,10]

print("Antes de adicionar o número com append():")
print(L)

print("Depois da adição com append():")
L.append(6)
print(L)

print("Antes da instrução extend():")
print(L)

print("Depois da instrução extend():")
L.extend(V)
print(L)
```

```
Antes de adicionar o número com append():
[1, 2, 3, 4, 5]
Depois da adição com append():
[1, 2, 3, 4, 5, 6]
Antes da instrução extend():
[1, 2, 3, 4, 5, 6]
Depois da instrução extend():
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

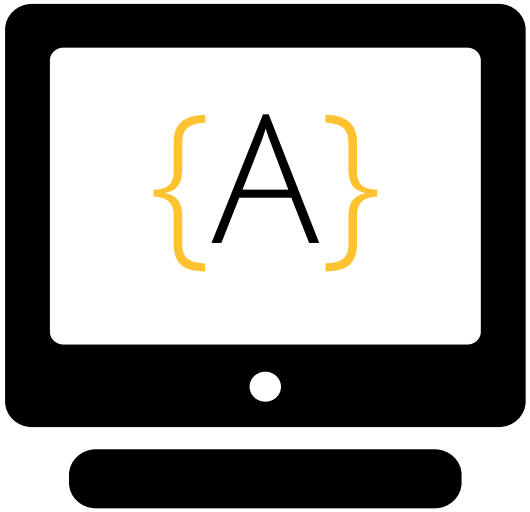
Listas

REMOVENDO ELEMENTOS EM UMA LISTA

Para remover dados da lista podemos utilizar a instrução *del* seguindo do nome da lista e o índice do elemento que se deseja remover.

```
L = [2,4,6,8,10]

print(L)

#Primeiro exemplo
print("Deletando o elemento do índice 2:")
del L[2]
print(L)

#Segundo exemplo
print("\nDeletando os elementos do índice 1 até o índice 3:")
del L[1:3]
print(L)
```

```
[2, 4, 6, 8, 10]
Deletando o elemento do índice 2:
[2, 4, 8, 10]

Deletando os elementos do índice 1 até o índice 3:
[2, 10]
```



Strings

O QUE SÃO?

Uma string é basicamente um grupo de caracteres. Ela pode ser uma frase ou uma palavra. Uma string é construída de várias strings menores, cada uma contendo um caractere. Dessa forma, assim como as listas, elas podem ser quebradas.

Strings

OPERAÇÕES COM STRINGS

Em geral, você não pode executar operações matemáticas com strings, mas é possível realizar alguns comandos com os operadores básicos + e *.

```
fruta = "banana"
tipo  = "bolo de "
print(tipo + fruta)
```

```
amanda@unito:~$ python3 ./Desktop/exemplo1.py
bolo de banana
amanda@unito:~$
```

```
saudacao = "Bom dia!"
print(saudacao*3)
```

```
amanda@unito:~$ python3 ./Desktop/exemplo2.py
Bom dia!Bom dia!Bom dia!
amanda@unito:~$
```

Strings

OPERADOR DE INDEXAÇÃO

Para acessar um caractere dentro da string, utilizamos o operador de indexação. Ele nos permite acessar o caractere por sua posição (valor do índice). Para isso, usamos o mesmo comando das listas. As posições também podem ser acessadas da direita para a esquerda, usando números negativos, onde -1 é o índice mais à direita.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
P	**R**	**O**	**J**	**E**	**T**	**O**		**P**	**R**	**O**	**G**	**R**	**A**	**M**	**E**	**N**	**I**	**N**	**A**	**S**
-21	-20	-19	-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Strings

COMPARADORES

Além do comparador de igualdade (==), podemos também verificar se existe um substring dentro de uma string. Para isso, utilizamos o comando *in.* Essa função retorna um valor booleano.

```
nome = "Marcos"
print('a' in nome)
print('b' in nome)
```

```
amanda@unito:~$ python3 ./Desktop/exemplo6.py
True
False
amanda@unito:~$
```